COGNITIONX EGYPT · 3 AUGUST 2026

AI-Native Development

From Writing Code to Owning the Full SDLC

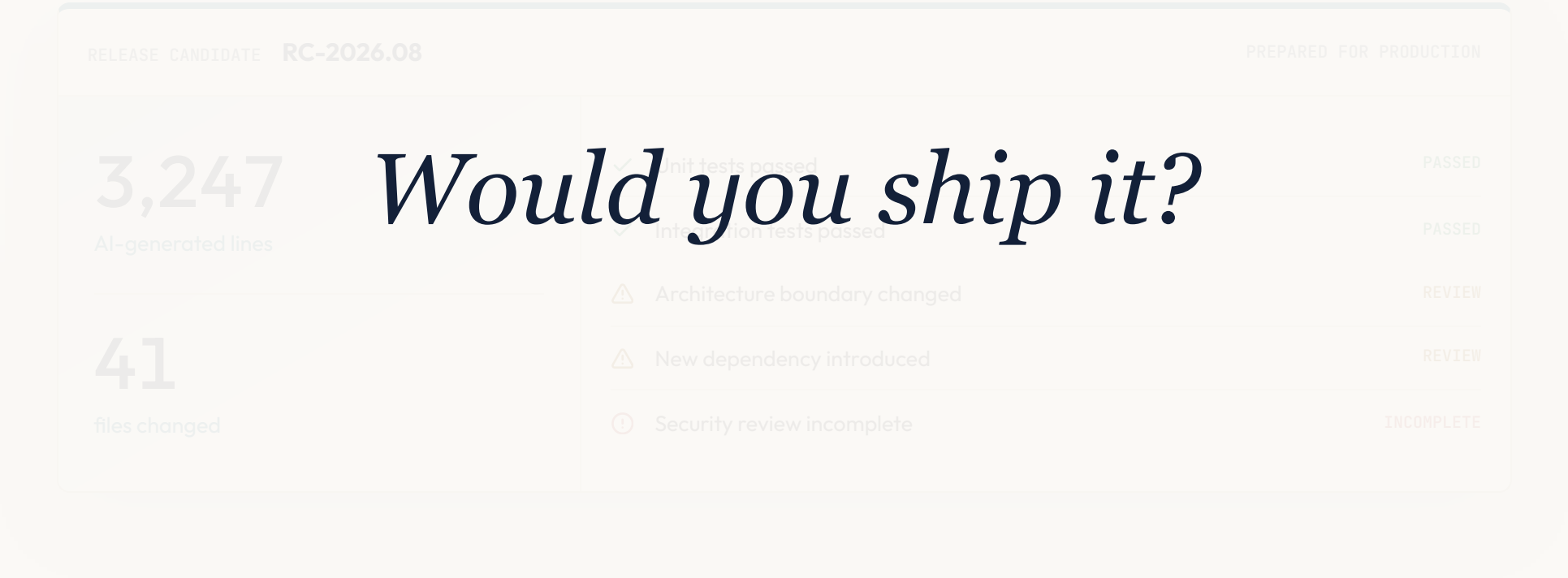
> *A role-evolution talk, not a tools talk.*

Direction · Constraints · Evidence · Outcomes

Hany Saad Senior Engineering Manager, ITWorx

You didn't write this code.

But your name is on the release.



Would you ship it?

AI can generate the implementation.

The engineer still owns the outcome.

THE VERIFICATION SHIFT

The Bottleneck Moved

Generating code

is becoming easier.

→

Knowing whether it deserves to ship

is becoming harder.

The new bottleneck is **verification.**

Do not surrender control to AI.
Move your control to a higher level.

Code → Behavior → Architecture → Risk → Outcome

You are the orchestrator—not the typist.

SAME PROMISE. DIFFERENT CONDITIONS.

+55.8%

FASTER

Controlled, self-contained task

Peng et al. · Microsoft Research / arXiv · 2023

−19%

SLOWER

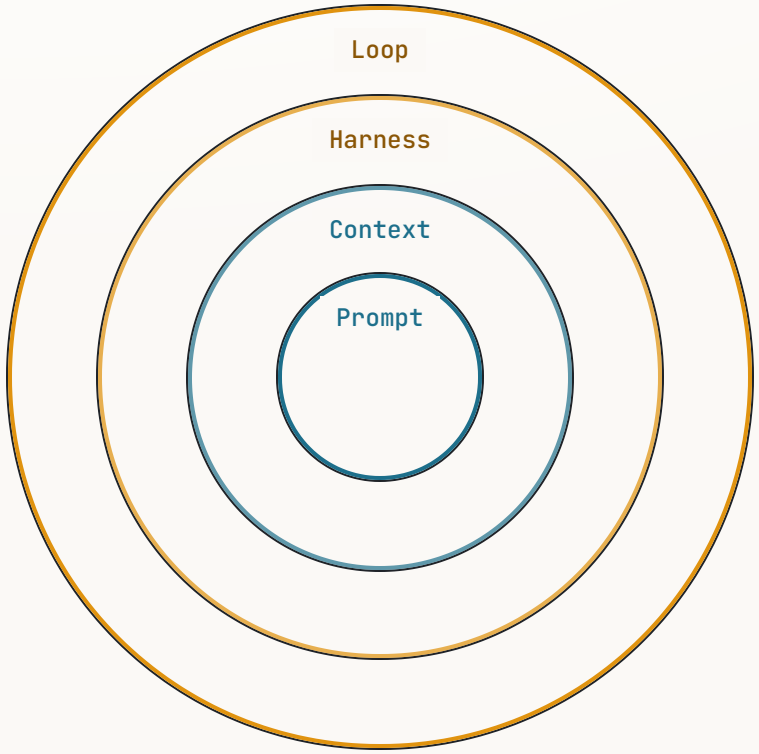
Experienced developers, mature repositories

METR · Mature repository study · 2025

Both results are real. What engineering conditions explain the difference?

A PRACTICAL WORKING MODEL

Prompt → Context → Harness → Loop

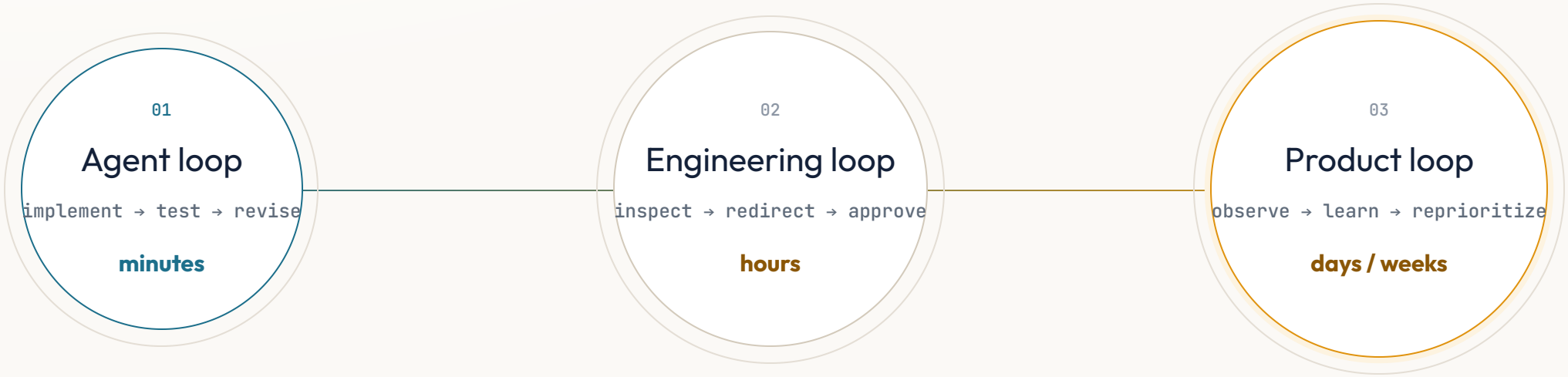


1	**Prompt**	What are we asking for?
2	**Context**	What does the agent need to know?
3	**Harness**	What can it access, execute, and change?
4	**Loop**	How does it verify, retry, stop, and escalate?

Each layer contains—and depends on—the one before it.

RESPONSIBILITY EXPANDS WITH LEVERAGE

Three Loops. Three Clocks.



The inner loop got fast. Your job moved outward.

THE LIFECYCLE REMAINS

The SDLC Stayed. The Verbs Changed.

LIFECYCLE STAGE	TRADITIONAL EMPHASIS	AI-NATIVE EMPHASIS
Discovery	Gather	**Frame**
Requirements	Document	**Specify**
Design	Diagram	**Constrain**
Implementation	Type	**Delegate**
Testing	Validate afterwards	**Define evidence first**
Review	Read the diff	**Assure the change**
Release	Ship	**Prove and recover**

AI changes where engineers apply judgment—not whether judgment is needed.

Critical and high-risk code still receives code-level inspection.

ACCOUNTABILITY HAS AN ADDRESS

What Owning the Full SDLC Actually Means

01	**Intent**	Problem, outcome, and the decision not to build
02	**Specification**	Acceptance criteria and edge cases
03	**Architecture**	Boundaries, contracts, and data flows
04	**Delegation boundary**	Scope, access, tools, and stopping conditions
05	**Evidence pack**	Tests, security, performance, and observability
06	**Release case**	Rollout, provenance, and rollback
07	**Production learning**	Telemetry, incidents, feedback, and improvement

Every artifact has an owner. AI is not that owner.

VALUE BEGINS BEFORE GENERATION

Start With Intent, Not Code

> *“Prevent session overbooking. If capacity is reached, offer a waitlist and notify organizers.”*
>
> Raw business request

- What counts as capacity?
- Who may override it?
- What personal data is exposed?
- What is the rollback behavior?
- Is check-in concurrent?
- How is the waitlist ordered?
- What happens when notification fails?

ACCEPTANCE CRITERIA Atomic capacity check · Authorized override · Stable waitlist order · Failure-tolerant notification · Auditable rollback

The engineer creates value before the first line is generated.

CHALLENGE THE PLAN BEFORE ACCEPTING THE WORK

Plan. Constrain. Delegate.

AGENT'S PROPOSED PLAN

01 Add waitlist storage

02 Update check-in endpoint

03 Add organizer notification

04 Update attendee UI

05 Add tests

HUMAN REVIEW

— Concurrency is missing

— Authorization boundary is unclear

— Notification failure must not reverse check-in

— No new dependency without approval

— Audit event required

Delegation contract `bounded tasks · affected interfaces · tests required · permissions allowed · stop / escalation conditions`

THE EVIDENCE IS THE DEMO

Verify the System, Not the Diff

01 Capacity acceptance tests ✓	02 Concurrent check-in test ✓	03 Authorization test ✓	04 Migration and rollback check ✓
05 Telemetry and alert ✓	06 Dependency and secret scan ✓	07 Manual review of critical logic ✓	

Requirement satisfied	Risks addressed	Evidence attached	**Ready for controlled release**

Explainable. Testable. Observable. Reversible.

CHANGE ASSURANCE STACK

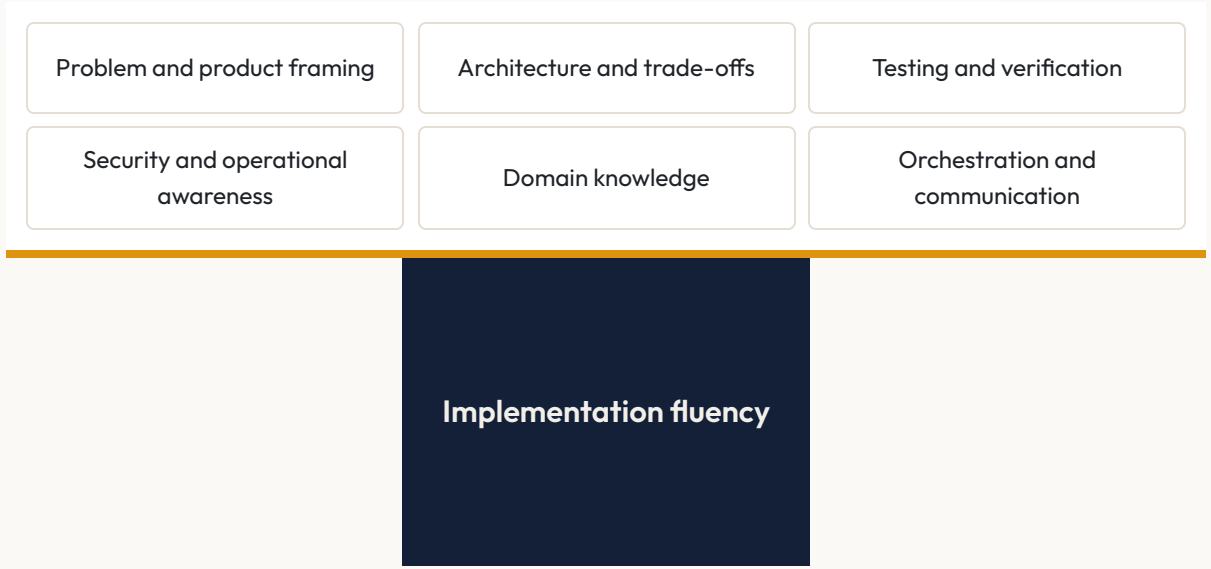
Code Review Is Necessary. It Is No Longer Sufficient.

Do not stop reviewing code.

Stop pretending code review alone proves the system is safe.

IMPLEMENTATION FLUENCY REMAINS ESSENTIAL

The Engineer Moves Up the Stack



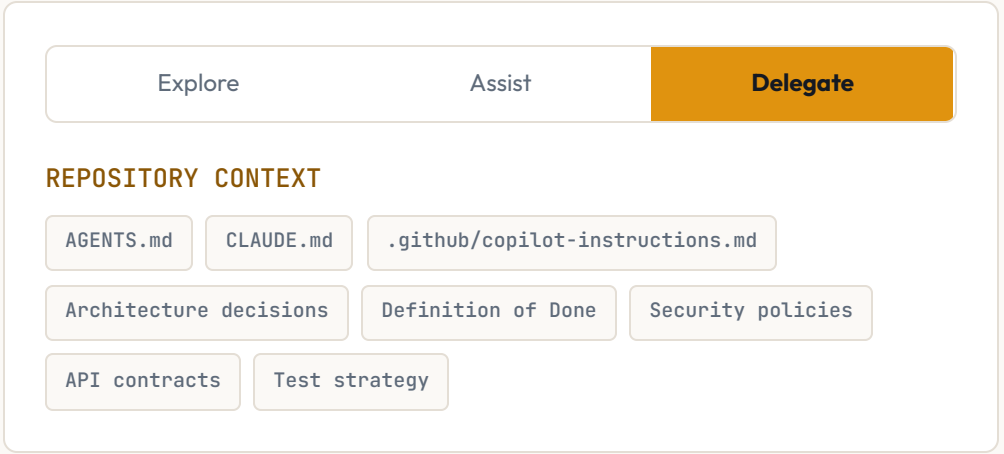
Junior	Learn concepts and critique generated work earlier
Senior	Scale judgment across more delivery
Architect	Make boundaries machine-readable
Manager	Redesign workflow, controls, and measurement

Skills once concentrated in senior roles are becoming earlier-career differentiators.

IMPROVE ONE WORKFLOW FIRST

Monday Morning: Five Moves

1. Choose one bounded, real task
2. Write acceptance criteria before invoking the agent
3. Make repository context explicit
4. Put automated evidence before human attention
5. Measure delivery—not generated lines



MEASURE DELIVERY cycle time · review effort · rework · escaped defects · change failure rate

OWN THE OUTER LOOPS

You are the orchestrator—not the typist.

The inner loop got fast. Own the outer loops.

AI-native development is not about producing more code.
It is about owning the system that turns intent into reliable outcomes.

Hany Saad
Senior Engineering Manager, ITWorx
linkedin.com/in/hanysaad